

From Langium Grammars to Blockly Editors A Restricted-Subset Code Generation Pipeline

Generative Software Engineering SS26

Md Mehedi Hasan and Julian Klüber

Bauhaus-Universität Weimar

Abstract. Textual grammars are an efficient way to define domain-specific languages (DSLs), but they require users to learn concrete syntax before they can write anything valid. Visual, block-based editors such as Google’s Blockly lower this barrier by letting users assemble programs from interlocking blocks instead of typing text, at the cost of someone having to hand-build a block definition and code generator for every DSL. We present a pipeline¹ that closes this gap automatically: given a Langium grammar restricted to a well-defined subset of constructs, our tool validates it, builds a small intermediate representation (IR) capturing what each grammar rule means as a user-facing input, and emits a ready-to-run Blockly block editor. We describe the pipeline’s four stages, two more complex code examples of translating Langium rules into Blockly blocks (cross-references rendered as a live, workspace-scanning dropdown field, and Langium’s unordered-group operator `&`), and validate the implementation against a 45-test unit/integration suite, a 12-test coverage suite, and a performance benchmark across synthetic grammars of up to 200 rules, all executed on the reference machine described in Appendix A.

1 Introduction

1.1 Background

Domain-specific languages let subject-matter experts express solutions in vocabulary close to their problem, rather than in a general-purpose programming language. Building a DSL from a textual grammar is well supported by modern language workbenches: Langium is an open-source TypeScript framework for implementing DSLs, generating a parser, an abstract syntax tree (AST), and a Language Server Protocol implementation directly from an `.langium` grammar file.

Textual syntax is not the only way to interact with a DSL, however. Block-based visual programming environments, popularised by Scratch and, in library form, by Google’s Blockly, let users construct programs by dragging and connecting graphical blocks. Blocks encode syntax structurally (a block that plugs into a statement socket *is* syntactically valid there by construction), which removes an entire class of errors a textual editor would otherwise have to catch after the

¹ https://github.com/JKlueber/grammar_to_blocks

fact, and makes DSLs approachable to users who are not yet comfortable typing code by hand.

1.2 Motivation

Building a Blockly editor for a DSL by hand means writing one block definition and one code-generator function per grammar rule, and keeping both in sync with the grammar as it evolves. This process is tedious, repetitive, and easy to let drift out of sync. Since a Langium grammar already declares, precisely, what each rule’s concrete syntax looks like (its sequence of keywords, assigned features, and sub-rule references), most of the information needed to derive a corresponding block definition is already present in the grammar itself.

This motivates the project’s central question: for a well-defined, restricted subset of Langium grammars, can a block editor, consisting of definitions, generator, and workspace, be derived automatically, rather than written by hand for every DSL?

2 Problem Statement

Given a Langium grammar G whose parser rules use only constructs from a supported subset S (Section 4.6 defines S precisely), the pipeline must produce:

- block definitions** one Blockly block type per parser rule in G , whose input fields/sockets correspond to that rule’s assigned features;
- a code generator** a function per block type that reconstructs a syntactically valid instance of G ’s concrete syntax from the values a user has entered into that block’s fields and connected inputs;
- a runnable editor** a Blockly workspace, pre-populated with a toolbox containing every generated block, that re-renders the generated DSL text live as the user edits the workspace.

If G uses a construct outside S , the pipeline must reject it before generating anything, reporting every offending construct and the rule it occurs in.

3 Example & Intuition

Before describing the pipeline in the abstract, it helps to trace one grammar all the way through it, since every stage’s job is easiest to see concretely: (1) Langium turns grammar text into an AST, (2) our Validator checks if all rules are part of the supported subset, (3) our IR builder turns that AST into a much simpler intermediate representation, (4) our target generator turns the IR into Blockly blocks, and (5) those blocks, once assembled by a user, are read back into valid DSL text.

Consider the bundled `Todo` grammar (Listing 1.1): a `Model` names itself and contains zero-or-more `Tasks`, and each `Task` has a name plus two keyword-alternative features, `priority` and `done`.

```

1 grammar Todo
2
3 entry Model:
4     'todo' name=ID '{'
5         items+=Task*
6     '}' ;
7
8 Task:
9     'task'
10    name=ID
11    priority=('low' | 'medium' | 'high')
12    done=('yes' | 'no')
13 ;

```

Listing 1.1. The bundled `todo_list.langium` example grammar

Step 1: Parsing to an AST. Langium’s own grammar services parse this text into a full abstract syntax tree: nested `ParserRule`, `Group`, `Assignment`, `Alternatives`, and `Keyword` nodes, each carrying the bookkeeping Langium needs for its own tooling (source positions, cross-reference resolution, and so on). This AST is precise, but it is also far more detailed than a code generator needs.

Step 2: Validating the grammar. The validator walks the grammar definitions to ensure every AST node type (such as `ParserRule`, `Assignment`, `Keyword`, and `Alternatives`) and cardinality (such as `*`) falls within the supported subset. It verifies that both `Model` and `Task` contain no unsupported constructs before continuing.

Step 3: Simplifying to an IR. The IR builder walks the AST and asks, for every rule and every one of its parts, one question: *what kind of user-facing input does this correspond to?* The answer is recorded as one `IRPart` per part of the rule (Section 4.2 gives the full picture). For `Task`, this produces four parts: a `keyword` for the literal `'task'`, a `field` for `name=ID` (a text box), and two `dropdowns` for `priority` and `done` (closed choices, one option per keyword). For `Model`, `items+=Task*` produces a `statement` part (Blockly’s stacking idiom for repetition) recording that `Task` blocks are what may be stacked there.

Step 4: Generating blocks from the IR. With the AST’s complexity already resolved into six plain `IRPart` kinds, turning the IR into Blockly JSON is now a simple, mechanical per-kind mapping: `field` becomes `field_input/field_number`, `dropdown` becomes `field_dropdown`, and `statement` becomes an `input_statement` whose “check” type ties it to the stacked rule’s own block. Listing 1.2 shows the result for `Task`: because `Task` is only ever repeated via `items+=Task*`, its block also receives a self-typed `previousStatement/nextStatement` of `"task"`, so several `Task` blocks can stack inside `Model`’s statement input.

Step 5: From blocks back to DSL text. A matching `forBlock['task']` function is generated alongside the block definition; it reads each field back out with `block.getFieldValue(...)` and splices the values into the rule’s original concrete syntax. Assembling one `model` block named `chores` with two `task`

```

1 {
2   "type": "task",
3   "message0": "task Name: %1 Priority: %2 Done: %3",
4   "args0": [
5     { "type": "field_input", "name": "NAME" },
6     { "type": "field_dropdown", "name": "PRIORITY",
7       "options": [ ["low", "low"], ["medium", "medium"],
8                   ["high", "high"] ] },
9     { "type": "field_dropdown", "name": "DONE",
10      "options": [ ["yes", "yes"], ["no", "no"] ] }
11   ],
12   "previousStatement": "task", "nextStatement": "task"
13 }

```

Listing 1.2. Generated block JSON for the Task rule (excerpt)

blocks stacked into its statement input, running the generator reconstructs the following text:

```

1 todo chores {
2   task dishes high no
3   task laundry low yes
4 }

```

Listing 1.3. Reconstructed DSL text for the workspace described above

This five-step round-trip (AST in, grammar validated, IR out, blocks generated, DSL text reconstructed) is exactly what every test in Section 5.1 exercises for the pipeline’s other bundled grammars, including the more involved cross-reference and unordered-group cases discussed in Sections 4.3 and 4.4.

4 Implementation

4.1 Pipeline Architecture

The pipeline (`generate_blockly/src/`) is organised as four sequential stages, each with a single, narrow responsibility:

1. `grammar-loader.js` parses the `.langium` file using Langium’s own grammar services (`createLangiumGrammarServices`) and throws if the document has any error-severity diagnostic.
2. `validator.js` walks every parser rule’s definition tree and collects every AST node type or cardinality that falls outside the supported subset, so a user gets one complete report instead of fixing violations one at a time.
3. `ir-builder.js` walks the (now validated) AST and produces an array of `RuleIR` objects: a rule name plus an ordered list of `IRParts`, the pipeline’s own grammar-agnostic vocabulary for “what does this piece of the rule mean as a block input.”

4. `blockly-ts-target.js` turns `RuleIR[]` into three TypeScript files:
 - `blocks.ts` (block definitions), `generator.ts` (the `forBlock` functions that reconstruct DSL text), and `main.ts` (workspace + toolbox wiring). Those are written directly into `blockly_app/src/`, ready for `npm run dev` to serve.

Separating stage 3 (grammar semantics) from stage 4 (Blockly-specific code generation) behind the small `IRPart` vocabulary is what let us extend the pipeline (Sections 4.3 and 4.4) by touching only one or two of the four stages at a time, rather than the whole codebase.

4.2 The Intermediate Representation

Section 3 showed the IR at work; this section gives its full shape. A `RuleIR` is just a rule name plus an ordered list of `IRParts`, one per keyword, feature, or sub-rule reference in that rule's definition (Listing 1.4). Every `IRPart` is a small, uniform record: a `kind` tag plus whatever extra fields that kind needs (which rule it refers to, its dropdown options, its cardinality). Crucially, neither stage 4 (`blockly-ts-target.js`) nor any of its helpers ever inspects a Langium AST node directly, everything downstream of `ir-builder.js` works purely in terms of these six kinds.

```

1  /**
2   * @typedef {Object} IRPart
3   * @property {"keyword"|"field"|"dropdown"|"value"
4   *           |"statement"|"reference"} kind
5   * @property {string}    [feature]      - grammar feature name
6   * @property {string}    [refRuleName] - referenced rule, if
7   *                                     any
8   * @property {boolean}   [optional]     - cardinality "?"
9   * @property {boolean}   [repeatable]   - cardinality "*" / "+" /
10  *                                     "+="
11  */

```

Listing 1.4. The `IRPart` shape (JSDoc typedef, abbreviated)

Each kind corresponds to exactly one shape of grammar syntax, chosen to match the Blockly input it will become:

keyword a literal token (`'recipe'`) with no user input at all: it renders as fixed text in the block's label.

field a `feature=ID` or `feature=INT` assignment: a text or number box the user types directly into.

dropdown an all-keyword **Alternatives** (`feature=('a'|'b'|'c')`): a closed choice, one option per keyword.

value a `feature=SomeRule` assignment: a socket that another block's output plugs into, for grammar rules nested by value rather than repeated.

statement a `feature+=Rule` assignment: a stacking socket, since Blockly's native idiom for repetition is stacking one block per list item rather than a single input holding many values.

reference a `feature=[Rule:TERMINAL]` cross-reference: pointing at an *already-declared* element by name rather than nesting a new one (Section 4.3).

4.3 Cross-References as a Live-Scanning Dropdown

A Langium cross-reference (`assignee=[Member:ID]`) means “point at an already-declared `Member` by name,” as opposed to `assignee=Member`, which nests a brand-new one. Blockly has no built-in widget for this, as a plain `field_dropdown`’s option list is fixed at block-definition time, while a cross-reference’s valid targets change as the user edits the workspace. We therefore ship `FieldReference`, a hand-written `Blockly.FieldDropdown` subclass (`blockly_app/src/reference-field.ts`, the one file in `blockly_app/src/` not regenerated by the pipeline) that overrides `getOptions()` to scan `workspace.getAllBlocks()` for every block of the referenced type and read each one’s declared name, live, every time the dropdown opens.

Wiring this up touches four files end to end: `ir-builder.js` records which rule a reference targets (and, via `computeNameFields`, which feature on *that* rule counts as its “name”; by convention, its first plain `feature=ID` field); `block-json-generator.js`’s `argBuilders.reference` emits `{"type": "field_reference", "referencesType": ..., "nameField": ...}`, falling back to a plain text field when the target rule has no name field to scan; `reference-field.ts` implements the field itself; and the generator in `blockly-ts-target.js` reads it back with the same `block.getFieldValue(...)` call used for any other field, unaware that the value came from a live dropdown rather than typed text. This trade-off is deliberate and documented: every declared name of the target type is offered regardless of where the referencing block sits on the workspace (Langium’s own cross-references support finer-grained scoping we do not model), and a renamed or deleted target simply stops matching, falling back silently to the first available option.

4.4 Extending the Pipeline: Unordered Groups

We added support for Langium’s `UnorderedGroup` construct (`a & b & c`: all of these, in any order), following the same three-step pattern the codebase already documents for extending it. First, `validator.js` adds `'UnorderedGroup'` to its allowed-type set and walks its `.elements` the same way it already walks a `Group`’s. Second, `ir-builder.js` gains a new `nodeHandlers.UnorderedGroup` entry (Listing 1.5) whose body is, deliberately, an exact copy of the existing `Group` handler.

```

1 UnorderedGroup(node, ctx) {
2   for (const e of node.elements)
3     visit(e, ctx);
4 },

```

Listing 1.5. The `UnorderedGroup` IR handler

The design rationale is what makes this extension worth reporting rather than a one-line diff: Blockly’s block UI is a static form (a block’s fields render in one fixed order, decided once at block-definition time) so there is no way to preserve `&`’s “any order is fine” flexibility inside a single block. Committing to

one fixed order (the grammar’s own declaration order) is still always *correct*, however: since the grammar rule itself does not care what order its parts appear in, always emitting them in one particular order is trivially one of the orders the grammar accepts. What is lost is only the *choice* a person typing the DSL by hand would have had, not the validity of what the block editor produces. Because no new `IRPart.kind` was introduced, stage 4 (`blockly-ts-target.js`) required no changes at all.

4.5 Tool Usage

The pipeline is driven from the command line:

```
1 npm install
2 node generate_blockly/src/parse.js generate_blockly/input/
  recipe.langium
3 npm run dev
```

Listing 1.6. Generating and running an editor for a grammar

The first command generates `blocks.ts/generator.ts/main.ts` into `blockly-app/src/`, overwriting the previous grammar’s output in place; the second serves the resulting Blockly workspace via Vite.

4.6 Limitations

The validator’s supported subset S is: `Group`, `UnorderedGroup`, `Alternatives`, `Assignment` (`=/+=`), `Keyword`, `RuleCall`, `CrossReference`, and cardinalities `?/*/+`/none. `Action` (Langium’s tree-rewriting/type-inference syntax, e.g. `{infer Type}`) is the one construct explicitly identified, but not yet implemented, as the next candidate for extension. Mixed `Alternatives` (keywords mixed with rule calls, or list-assignments to more than one shape) fall back to visiting only the first branch, emitting a warning rather than failing outright, so the pipeline always produces *something*. Cross-reference scoping and unordered-group ordering choice, discussed above, are both deliberate simplifications rather than oversights, each documented at the point in the code where the simplification is made.

5 Evaluation

We report only the most important outcomes here; the full test and benchmark output collected from the reference machine is reproduced in Appendix A.

5.1 Validation

Correctness is checked with Node’s built-in test runner. Running `npm test` executes 45 unit/integration tests across `tests/validation/` (one file per pipeline module plus `roundtrip.test.js`, which compiles the actually-generated `blocks.ts/generator.ts` to real ES modules, loads them against the real

`blockly` npm package, builds a headless `Blockly.Workspace`, wires up blocks the way a person dragging them in the UI would, and asserts on the DSL text `Blockly`’s own generator reconstructs). On the reference machine this suite completed in 5.66 s with 45/45 passing, 0 failing.

A second, separate suite (`npm run test:eval`) checks breadth rather than per-module correctness: 12 tests confirm that (i) every one of the 6 bundled example grammars and 3 fixtures produces the outcome documented for it (Table 2 in the appendix reproduces the full outcome matrix), (ii) all six documented `IRPart.kind` values are exercised by at least one bundled grammar, and (iii) the one construct the supported-subset table names as unsupported (`Action`) is in fact rejected. This suite completed in 1.70 s with 12/12 passing, 0 failing. Together, the two suites give 57/57 tests passing on the current implementation, including the `UnorderedGroup` extension from Section 4.4.

5.2 Performance Cost

We benchmarked each pipeline stage (`npm run test:perf`) against both the 6 bundled example grammars and synthetic grammars of increasing size (1 to 200 rules, generated by `synthetic-grammar.js`), 7 timed repetitions after 2 warm-up runs each, on the single reference laptop described in Appendix A (Table 1). Full per-stage tables are given in Appendix A (Tables 3 and 4); the headline trends are:

- `validate`, `buildIR`, and `generate` stay sub-millisecond to low-single-digit-millisecond even at 200 rules (0.50 ms, 0.28 ms, and 7.27 ms median, respectively), confirming they are simple, roughly-per-rule loops.
- `loadGrammar` dominates total pipeline time end to end, from 31 ms at 1 rule up to 241 ms at 200 rules. As documented directly in the code, this is because it calls `createLangiumGrammarServices()` fresh on every invocation rather than reusing a cached services object; the cost is largely independent of grammar size, which is exactly why `load` scales the most sub-linearly of the four stages (Table 4).

This is negligible for the pipeline’s intended one-shot CLI usage, but is worth caching if the pipeline were ever driven from a long-running process converting many grammars per process lifetime.

6 Discussion & Conclusion

We implemented a four-stage pipeline that turns a restricted-subset Langium grammar into a working Blockly editor. Beyond the base pipeline, we included two features: cross-references rendered as a live, workspace-scanning dropdown, and Langium’s unordered-group operator.

The clearest direction for future work is closing the one remaining gap the supported-subset table names explicitly: `Action` support, which would additionally require deciding how Blockly should represent Langium’s type-inference/tree-rewriting semantics. This is a substantially different kind of construct from those

we currently support, since it does not correspond to a single, obvious block input the way an assignment or a keyword does. Scoped cross-references (restricting a reference's live-scanned options to blocks actually reachable from the referencing block's position on the workspace, rather than every declared instance workspace-wide) and an "optional input" mutator for ?-cardinality fields are two further, smaller extensions the current architecture is already well positioned to support. On the performance side, caching the Langium grammar services object across invocations (Section 5) is the one concrete, already-identified optimisation that would meaningfully change the pipeline's scaling profile if it were ever run outside a one-shot CLI context.

A Full Test and Benchmark Results

This appendix reproduces, in full, the raw output referenced in Section 5, collected in a single `npm run test:all` run (validation suite, coverage suite, performance benchmark) plus a hardware inventory of the reference machine.

A.1 Reference Machine

Table 1. Reference machine used for every timing measurement in this appendix

Component	Description
CPU	Intel Core i5-1135G7 @ 2.40 GHz (11th Gen, Tiger Lake-LP)
RAM	16 GiB LPDDR4, 4267 MHz
Storage	Samsung MZVL2512HCJQ-00BT7, 512 GB NVMe SSD
GPU	Intel Iris Xe Graphics (integrated)
OS	Ubuntu (Linux), Node.js (<code>node:test</code> runner)

A.2 Validation Suite (`npm test`) – 45 Tests

All 45 tests, spanning `grammar-loader.test.js`, `validator.test.js`, `ir-builder.test.js`, `block-json-generator.test.js`, `blockly-ts-target.test.js`, and `roundtrip.test.js`, passed. Total wall-clock time: 5655.58 ms (45 pass, 0 fail, 0 cancelled, 0 skipped). Two informational Blockly console warnings (`Block definition "model"/"task" overwrites previous definition`) were emitted by the round-trip tests as an expected side effect of repeatedly calling `defineBlocks()` against a shared Blockly type registry across test cases in the same process; they do not indicate a test failure.

A.3 Coverage Suite (`npm run test:eval`) – 12 Tests

All 12 tests passed in 1702.52 ms (12 pass, 0 fail). Table 2 reproduces the coverage summary table the suite itself prints, confirming every bundled/fixture grammar reaches exactly the pipeline stage (load, validate, or full success) documented for it.

The suite additionally confirmed that all six documented `IRPart.kind` values (`keyword`, `field`, `dropdown`, `value`, `statement`, `reference`) are exercised by at least one of the six `pipeline-succeeds` grammars above, and that the one construct explicitly documented as unsupported (`Action`) is in fact rejected by `validateGrammar`.

Table 2. Coverage summary: expected vs. actual pipeline outcome per grammar

Grammar	Expected	Actual	Note
<code>todo_list.langium</code>	pipeline-succeeds	pipeline-succeeds	–
<code>state_machine.langium</code>	pipeline-succeeds	pipeline-succeeds	–
<code>recipe.langium</code>	pipeline-succeeds	pipeline-succeeds	–
<code>address_book.langium</code>	pipeline-succeeds	pipeline-succeeds	–
<code>grammar.langium</code>	pipeline-succeeds	pipeline-succeeds	–
<code>cross_references.langium</code>	pipeline-succeeds	pipeline-succeeds	–
<code>unordered-group.langium</code>	pipeline-succeeds	pipeline-succeeds	–
<code>syntax-error.langium</code>	fails-at-load	fails-at-load	“Grammar contains errors:”
<code>action.langium</code>	fails-at-validate	fails-at-validate	“Grammar uses unsupported constructs:”

A.4 Performance Benchmark (`npm run test:perf`)

Table 3 gives per-stage median timings for the six bundled example grammars; Table 4 gives the same for synthetic grammars of increasing size (`synthetic-grammar.js`, 1–200 rules). Both were produced with 7 timed repetitions after 2 warm-up runs per grammar.

Table 3. Bundled example grammars: median stage time (ms)

Grammar	Rules	Load	Validate	Build IR	Generate	Total
<code>todo_list.langium</code>	2	58.66	0.01	0.01	0.13	58.81
<code>state_machine.langium</code>	2	36.11	0.01	0.01	0.07	36.20
<code>recipe.langium</code>	3	34.75	0.01	0.01	0.15	34.92
<code>address_book.langium</code>	4	37.11	0.01	0.02	0.11	37.25
<code>grammar.langium</code>	7	41.67	0.02	0.03	0.35	42.08
<code>cross_references.langium</code>	3	33.43	0.01	0.01	0.11	33.56

As noted in Section 5, `loadGrammar` re-creates Langium’s grammar services on every call (see `generate.blockly/src/grammar-loader.js`), which is why it dominates absolute total time despite scaling the most sub-linearly of the four stages; `validate`, `buildIR`, and `generate` are all simple per-rule loops and remain in the sub-millisecond-to-low-single-digit-millisecond range even at 200 rules.

Table 4. Synthetic grammars of increasing size: median stage time (ms)

Grammar size	Rules	Load	Validate	Build IR	Generate	Total
1 rule	2	31.23	0.01	0.01	0.07	31.31
5 rules	6	33.97	0.04	0.05	0.48	34.53
10 rules	11	40.13	0.07	0.09	0.90	41.18
25 rules	26	63.60	0.14	0.20	1.22	65.16
50 rules	51	95.57	0.08	0.23	4.28	100.16
100 rules	101	138.85	0.39	0.35	5.08	144.67
200 rules	201	240.58	0.50	0.28	7.27	248.63